Reporte

Nombre: Equipo AulaPlus (Caso 08): Ángel Adrián Velez Contreras 100% Alonso Morales Quintanilla 70% Jairany López Ramos 100%	**Matrícula:** 2145646 2145757 2145407
Nombre del curso: Componentes en Lógica de Negocios	**Nombre del profesor:** Miguel Ángel Gómez Marroquín
Unidad: 3	**Actividad:** Diseño arquitectural del sistema con IA, Informe de arquitectura de AulaPlus **Link del repositorio:** https://github.com/jyrnso0/Aulaplus
Fecha: 13/10/2026	

Bibliografia (Formato APA)

Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley.
Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.
Martin, R. C. (2003). Agile Software Development: Principles, Patterns, and Practices. Prentice Hall.
Nygard, M. (2011). Documenting architecture decisions. Cognitect Blog.
Young, G. (2010). CQRS Documents. https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf

Informe de arquitectura de AulaPlus

Diseño arquitectural con Domain Model, Service Layer, Repository, Unit of Work y CQRS

Objetivo

Presentar la arquitectura de AulaPlus, el marketplace de tutorías, de modo que las reglas del negocio levantadas en la Unidad 2 queden organizadas en un dominio, una capa de aplicación y una infraestructura, con cada decisión justificada, trazable a una regla y revisada con SOLID. El informe reúne las etapas 2 a 8 de la actividad y cierra con las decisiones que todavía debe validar el negocio.

1. Alcance

El caso es **AulaPlus (Caso 08)**. Se trabaja sobre los cuatro casos de uso de las capturas: CU-01 reserva y confirmación de pago, CU-02 registro de disponibilidad y prevención de traslapes, CU-03 cancelación tardía de una sesión y CU-04 bloqueo del cierre de una disputa

sin evidencia. Para cubrir las 20 historias de usuario, la Service Layer añade los casos complementarios CU-05 a CU-13 (sesión, ausencias, acuerdos, calificaciones, liquidación y consultas).

- **Reglas que gobiernan el diseño:** BR-0001 (pago en 15 minutos), BR-0004 (penalización del 50% con menos de 4 horas), BR-0005 (comisión 20% y 80% al tutor), BR-0006 (sin traslapes), BR-0007 (disputa dentro de 24 horas), BR-0011 (resolución en 48 horas), BR-0018 (suspensión por ausencias) y BR-0020 (no se cierra una disputa sin evidencia).
- **Dentro del alcance:** decisiones de arquitectura, diagramas de clases y de secuencia, contratos de los servicios y trazabilidad entre necesidades, reglas, decisiones y componentes.
- **Fuera del alcance:** implementación completa, integración real con un banco, interfaz de usuario y cualquier política que el negocio no haya definido. Los vacíos del BRD se registran como pendientes y no se inventan.

2. Comparación de patrones de lógica de negocio (Etapa 2)

Las reglas de AulaPlus dependen del estado de la sesión, del momento en que ocurre la acción y de quién la ejecuta, y deben comportarse igual desde la app del estudiante, la del tutor, Soporte y los procesos automáticos. Esa condición decidió la comparación.

Patrón	Ventajas	Riesgos en AulaPlus	Resultado
Transaction Script	Desarrollo inicial rápido; simple para una operación aislada.	La misma regla se duplica en cada script (pagar, cancelar, resolver disputa). Un script que olvide validar la evidencia permitiría cerrar una disputa sin respaldo.	Descartado
Table Module	Adecuado para reportes y operaciones masivas sobre tablas.	La lógica queda atada a la estructura de la base de datos; reglas combinadas (ausencia, reembolso, suspensión) quedan repartidas entre módulos que no conocen el estado de la sesión.	Descartado
Domain Model	Cada regla vive junto al dato que protege; los estados inválidos se impiden en el propio objeto.	Mayor esfuerzo de diseño inicial, curva de aprendizaje y riesgo de un modelo anémico si las reglas se escapan a los servicios.	**Elegido**

Decisión. Domain Model para todo lo que cambia estado: reserva, pago, cancelación, sesión, disputa y suspensión de tutores. Los Application Services solo coordinan. Las consultas de solo lectura (dashboard semanal y perfil público del tutor) quedan fuera del dominio y se resuelven con un modelo de lectura propio, lo que enlaza con la sección 7. La decisión se verifica con la revisión SOLID de la sección 8, que vigila que no aparezca un modelo anémico.

3. Mapa de dominio (Etapa 3)

El mapa se construyó con el lenguaje del Glosario de la Unidad 2. No se convirtió cada tabla en entidad: solo se modelaron los conceptos con identidad, ciclo de vida o reglas propias. Cinco aggregates son límites de consistencia, y entre ellos solo se guarda el identificador del otro.

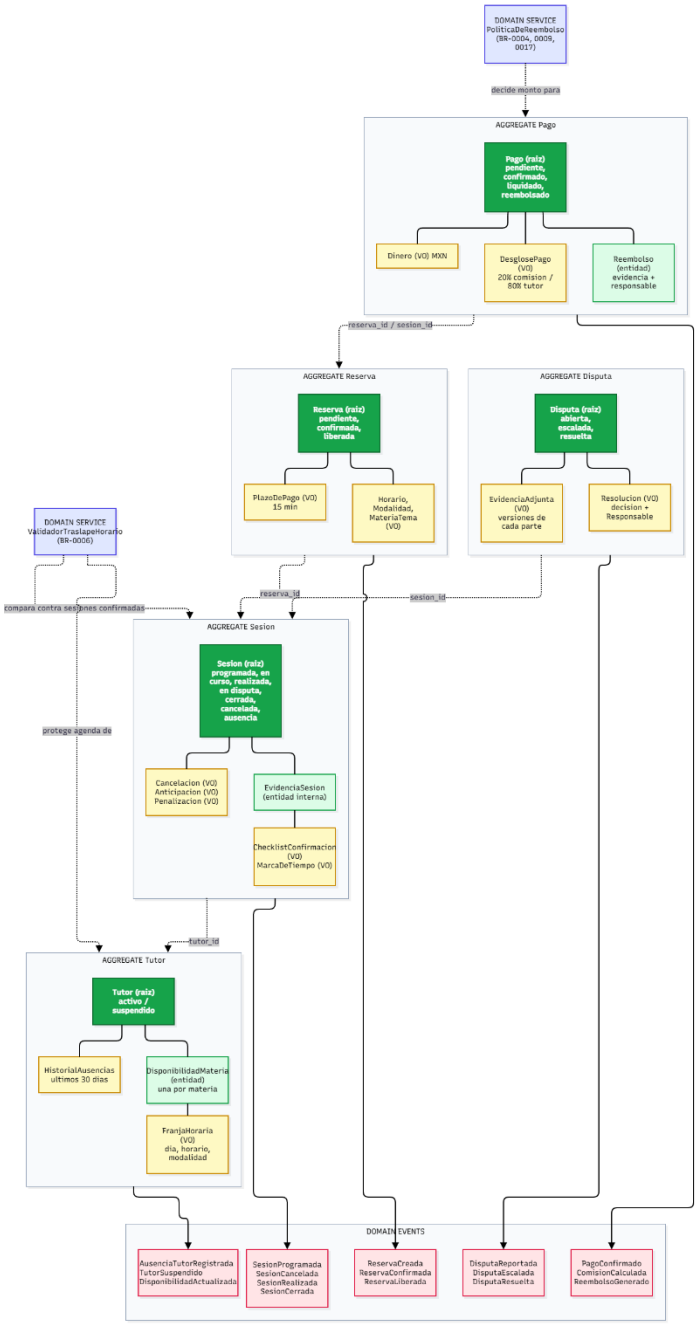


Figura 1. Mapa de dominio de AulaPlus: aggregates, entidades, value objects, servicios de dominio y eventos.

Aggregate (raíz)	Contiene	Invariantes que protege	Eventos
Reserva	PlazoDePago, Horario, importe	No se confirma después de 15 minutos; si vence, el horario se libera (BR-0001). Al confirmarse da origen a una sesión.	ReservaCreada, ReservaConfirmada, ReservaLiberada
Sesión	Cancelacion (con Penalizacion), evidencia y checklist	Penalización del 50% con menos de 4 h (BR-0004); no se da por realizada sin checklist de ambos (BR-0014); disputa solo dentro de 24 h (BR-0007); cierre a las 48 h (BR-0002).	SesionProgramada, SesionCancelada, SesionRealizada, SesionCerrada, AusenciaTutorRegistrada, DisputaReportada
Tutor	DisponibilidadMateria, historial de ausencias	Disponibilidad independiente por materia (BR-0015); suspensión con más de 2 ausencias en 30 días (BR-0018).	TutorSuspendido, DisponibilidadActualizada
Pago	Dinero (MXN), DesglosePago, Reembolso	Comisión fija del 20% (BR-0005); todo reembolso lleva evidencia y responsable (BR-0017); un pago reembolsado no se liquida.	PagoConfirmado, ComisionCalculada, ReembolsoGenerado
Disputa	EvidenciaAdjunta, Resolucion	No se resuelve sin evidencia adjunta (BR-0020); solo Soporte la resuelve; plazo de 48 h (BR-0011).	DisputaEscalada, DisputaResuelta

Servicios de dominio. ValidadorTraslapeHorario compara un horario contra las sesiones confirmadas del tutor (BR-0006), y PoliticaDeReembolso decide cuánto se devuelve según el motivo y quién cancela (BR-0004, BR-0009, BR-0017). Ninguno cabe en una sola entidad. La Etapa 4 agrega PoliticaDeAsistencia y PoliticaDeLiquidacion, también sin estado.

Decisiones de modelado. Reserva y Sesión son aggregates separados porque tienen ciclos de vida distintos. El traslape se revalida al confirmar el pago, porque dos reservas pendientes pueden pedir el mismo horario y gana la primera que paga. La Calificación queda fuera de Sesión, y el dashboard y el promedio público quedan fuera del dominio.

4. Service Layer (Etapa 4)

La capa de aplicación expone una operación por caso de uso ejecutable. Autoriza al actor, carga los aggregates, invoca el dominio, delimita la transacción y devuelve un DTO. No contiene porcentajes, umbrales ni transiciones de negocio.

Capa	Responsabilidad	Exclusiones
Presentación	Recibe datos y presenta DTOs y errores.	No calcula comisiones ni decide ausencias.
Aplicación	Autoriza al actor, carga agregados, invoca dominio, delimita la transacción y devuelve el resultado.	No contiene porcentajes, umbrales ni transiciones de negocio.
Dominio	Reserva, Sesion, Tutor, Pago y Disputa; objetos de valor y políticas.	No depende de HTTP, interfaz, proveedor de cobro ni SQL.
Infraestructura	Repositorios, reloj, auditoría, transacciones, planificador y notificaciones.	No decide si una disputa tiene sustento ni quién paga.

Application Service	Operaciones	Colabora con
ReservaApplicationService	reservarTutoria, confirmarPagoReserva, liberarReservasVencidas	ReservaRepository, SesionRepository, PagoRepository, ValidadorTraslapeHorario, PasarelaPagoExterna
TutorApplicationService	registrarDisponibilidad	TutorRepository
SesionApplicationService	iniciarSesion, confirmarChecklist, finalizarSesion, cancelarSesion	SesionRepository, PoliticaDeReembolso
DisputaApplicationService	reportarDisputa, resolverDisputa	DisputaRepository, SesionRepository

Contrato común. Cada comando recibe un DTO validado, un idOperacion y el contexto autenticado, y devuelve un DTO inmutable o un error tipado: NO_AUTORIZADO, PLAZO_VENCIDO, HORARIO_NO_DISPONIBLE, EVIDENCIA_REQUERIDA, CONFLICTO_VERSION, POLITICA_PENDIENTE y FALLO_PERSISTENCIA. La misma clave con el mismo contenido devuelve el resultado ya confirmado, así que un reintento no duplica cobros ni reembolsos.

Reglas que se movieron al dominio. Durante la revisión, el plazo de cambio de modalidad (CU-09) pasó a Sesion y la elegibilidad de liquidación (CU-11b) pasó a PoliticaDeLiquidacion, para que SesionApplicationService solo coordine y persista.

5. Repository (Etapa 5)

Las interfaces viven en el Dominio y operan solo sobre raíces de aggregate. Sus métodos expresan intención de negocio y no exponen SQL ni uniones de tablas. Las implementaciones (SqlReservaRepository, SqlSesionRepository, SqlTutorRepository, SqlPagoRepository, SqlDisputaRepository) viven en Infraestructura y mapean los value objects a columnas mediante un ORM.

Repository	Necesidad del dominio	Métodos
ReservaRepository	Recuperar una reserva pendiente para validar su pago o liberarla si vence el plazo de 15 minutos.	obtenerPorId, guardar, pendientesVencidas(ahora)
SesionRepository	Consultar las sesiones confirmadas de un tutor para prevenir traslapes y registrar la evidencia.	obtenerPorId, obtenerSesionesConfirmadasPorTutor, guardar
DisputaRepository	Gestionar el expediente y asegurar que no se cierre sin evidencia.	obtenerPorId, actualizar
PagoRepository	Gestionar el desglose del cobro (20/80), penalizaciones y reembolsos.	porReserva, guardar
TutorRepository	Administrar disponibilidad por materia y ausencias para la suspensión (BR-0018).	obtenerPorId, guardar

6. Unit of Work (Etapa 5)

El Unit of Work trata las operaciones de un caso de uso como una transacción atómica: o se completan todas, o no se aplica ninguna.

- **Dónde inicia:** en la capa de aplicación, con uow.begin() antes de cargar los objetos. Así se adquiere la exclusión de agenda del tutor y se evitan traslapes y dobles cobros.
- **Qué incluye:** todas las llamadas guardar() o actualizar() de los repositorios que modifiquen estado. Al terminar con éxito se llama uow.commit().
- **Qué ocurre ante un error:** si se viola una regla (por ejemplo BR-0020), falla la red o hay conflicto de concurrencia, se ejecuta uow.rollback() y la base queda intacta.
- **Después del commit:** los eventos de dominio se entregan mediante un outbox local, de modo que un evento no confirmado nunca llega a quien lo escucha.

Ejemplo (CU-01b, confirmar el pago). Una sola transacción actualiza tres aggregates: Reserva pasa a Confirmada si no venció, Pago registra el importe validado y el desglose 20/80, y nace una Sesion Programada. Si el servidor falla al guardar la sesión, la reserva y el pago se revierten. El diagrama de secuencia DS-02 (sección 9) muestra este flujo.

7. Separación de Commands y Queries, CQRS (Etapa 6)

AulaPlus aplica CQRS lógico sobre la misma base de datos: los Commands cambian estado pasando por el Domain Model, el Repository y el Unit of Work, y las Queries leen de un modelo de lectura sin cargar aggregates. No se crean bases separadas, porque no hay un requisito de concurrencia que lo justifique.

Tipo	Nombre	Entrada	Resultado
Command	ConfirmarPagoReservaCommand	idReserva, datosPago	Reserva Confirmada, Pago confirmado y Sesión programada, en un solo commit (BR-0001, BR-0006).

Tipo	Nombre	Entrada	Resultado
Command	ResolverDisputaCommand	idDisputa, idResponsableSoporte, decision, justificacion, evidenciaAdjunta	Disputa Resuelta; si procede, reembolso generado (BR-0020, BR-0011, BR-0017).
Query	ConsultarDashboardSemanalQuery	rangoFechas, idAdministrador	DashboardSemanalViewDTO: comisiones, cancelaciones, disputas y calificación promedio.
Query	ConsultarPerfilTutorQuery	idTutor	PerfilTutorViewDTO: calificación promedio pública, materias y disponibilidad.

Los Commands entran por ReservaApplicationService.confirmarPagoReserva y DisputaApplicationService.resolverDisputa. Los handlers de Query dependen solo de DashboardReadModel y PerfilTutorReadModel, que se implementan en Infraestructura con consultas o vistas sobre las mismas tablas, y no usan Repository ni Unit of Work. Los DTO de lectura se diseñan para la pantalla: no tienen métodos ni reglas y dejan fuera a propósito el historial de ausencias, la suspensión, los datos de pago y las calificaciones individuales.

Qué no necesita la separación. El CRUD simple sin reglas de negocio se queda como está. **Costo aceptado:** dos modelos que mantener y proyecciones de lectura que pueden actualizarse después del commit, por lo que deben mostrar su última actualización.

8. Revisión SOLID (Etapa 7)

Principio	Evidencia en el diseño	Cambio o decisión
S · Responsabilidad única	En la Etapa 4, CU-09 y CU-11b hacían que el servicio revalidara plazos y consultara la elegibilidad en Sesión, Disputa y Pago. Cambiaría con BR-0013, BR-0005 o BR-0017.	El plazo pasa a Sesion y AcuerdoDeModalidad. La elegibilidad pasa a PoliticaDeLiquidacion. El reembolso queda en PoliticaDeReembolso. El servicio solo coordina y persiste.
O · Abierto/Cerrado (propuesta)	PoliticaDeReembolso decide según la causa: cancelación tardía, ausencia del tutor o del estudiante, o corte de conexión (BR-0004, 0008, 0009, 0010). Cada causa nueva obligaría a editar la clase.	Se propone definirla como interfaz con una implementación por causa. Se marca como propuesta porque el diagrama actual muestra una sola clase.
L · Sustitución de Liskov	Cinco repositorios Sql* implementan la interfaz de su aggregate.	Los contratos usan solo tipos del dominio, así que una implementación en memoria para pruebas puede sustituir a la SQL.

Principio	Evidencia en el diseño	Cambio o decisión
I · Segregación de interfaces	Los handlers de lectura solo necesitan consultar y, si usaran Repository, dependerían de métodos de escritura como guardar.	Las lecturas usan DashboardReadModel y PerfilTutorReadModel. Hay un Repository por aggregate, no una interfaz gigante.
D · Inversión de dependencias	ReservaApplicationService necesita cobrar y guardar, lo que podría atarlo a un proveedor de pagos y a SQL.	Depende de PasarelaPagoExterna, de los Repository y de UnitOfWork. Las implementaciones viven en Infraestructura.

9. Diagramas (Etapa 8)

Los tres diagramas representan la misma arquitectura. Los participantes de los diagramas de secuencia son los componentes del diagrama de clases, agrupados por capa.

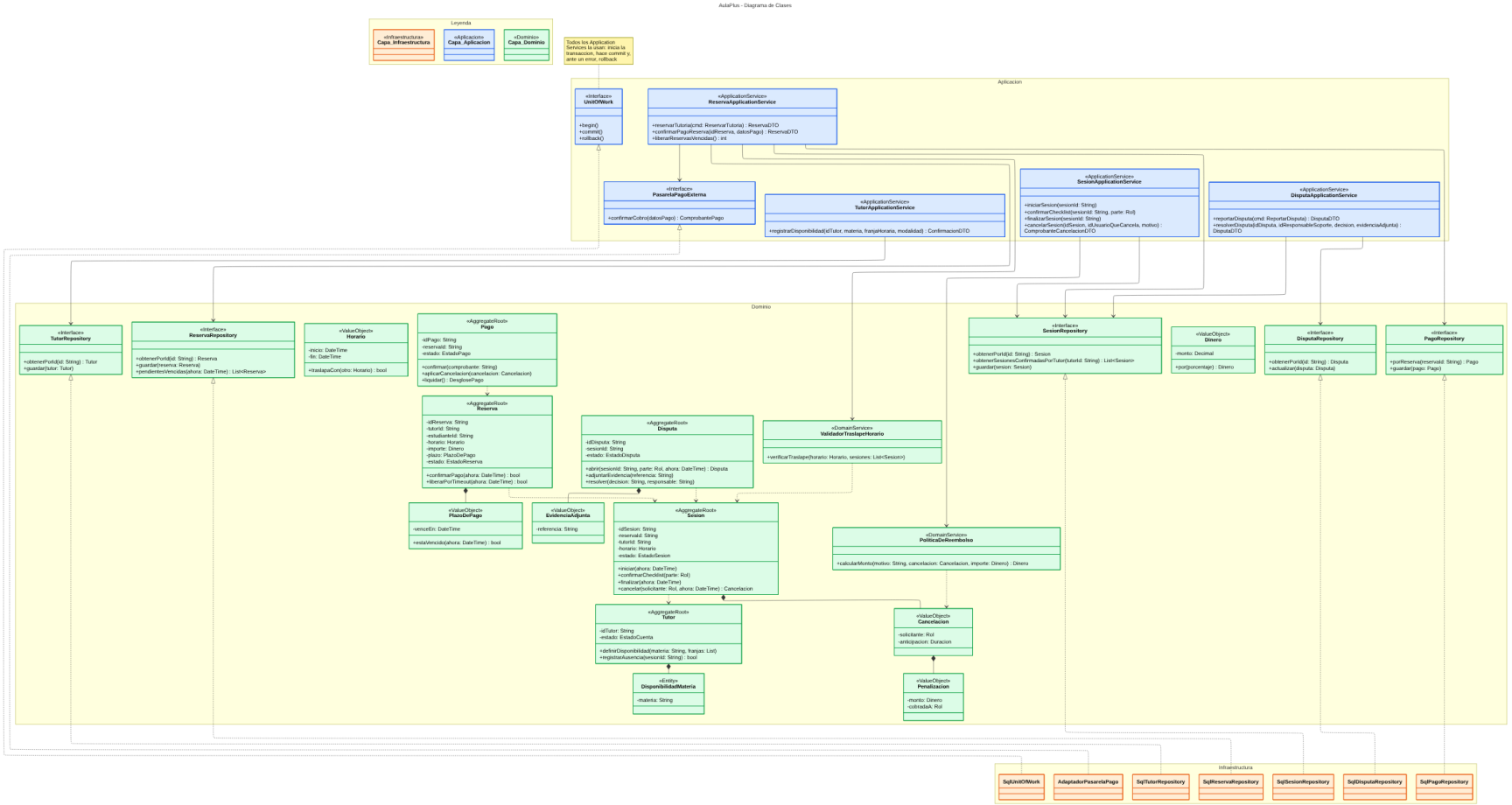


Figura 2. Diagrama de clases de AulaPlus (Aplicación, Dominio e Infraestructura).

Diagrama de secuencia DS-02 · flujo principal

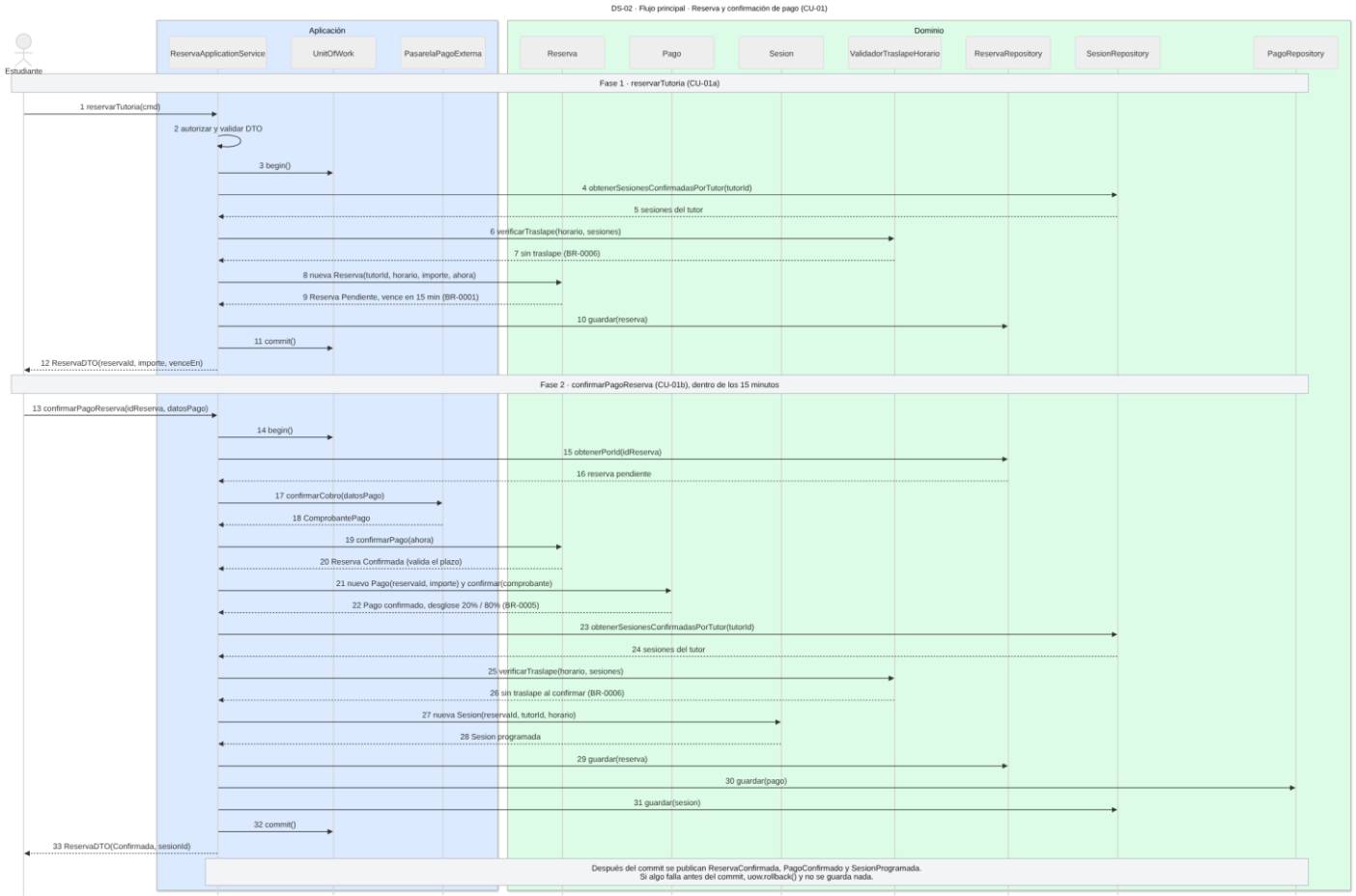


Figura 3. DS-02, reserva y confirmación de pago (CU-01). Fase 1 crea la reserva con vencimiento a 15 minutos; fase 2 cobra, confirma, revalida el traslape y programa la sesión en una sola transacción.

Diagrama de secuencia DS-01 · flujo con regla y alternativa

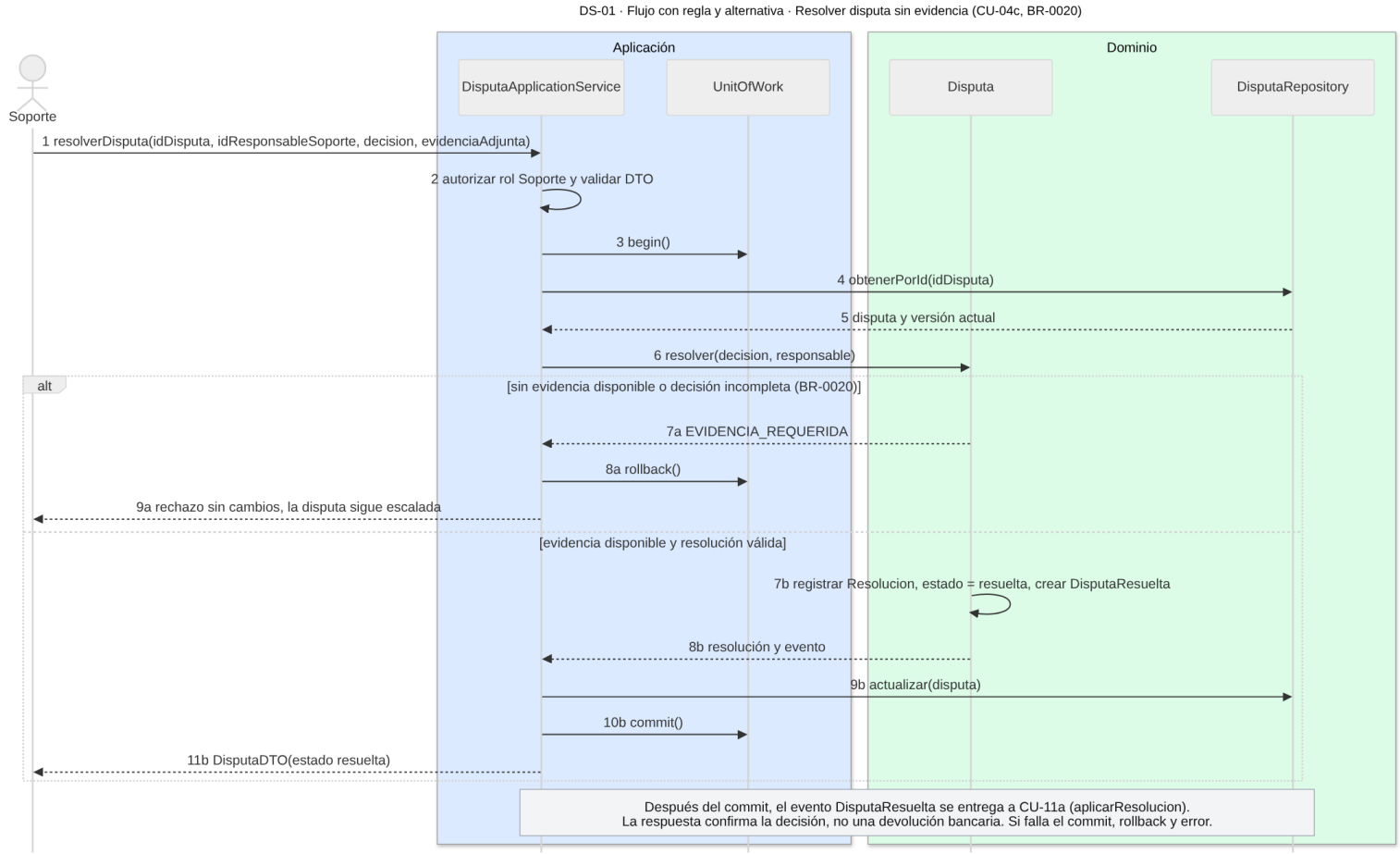


Figura 4. DS-01, resolución de una disputa (CU-04c). La alternativa alt aplica BR-0020: sin evidencia se devuelve EVIDENCIA_REQUERIDA y se hace rollback; con evidencia y resolución válida se actualiza y se hace commit.

10. Registros de decisiones arquitectónicas (ADR)

Los ADR conservan su historial. ADR-01 documenta la selección expresa de Domain Model; ADR-02 a ADR-07 son propuestas técnicas sujetas a revisión del equipo y ninguna convierte un vacío del negocio en una política aprobada.

ADR	Decisión	Alternativa descartada	Reglas
01 · Domain Model	Encapsular el comportamiento en raíces, entidades, value objects y servicios de dominio.	Transaction Script y Table Module.	Todas las BR; CA-04, 05, 20
02 · Service Layer y Unit of Work	Monolito modular con servicios de aplicación delgados, un repositorio por raíz, unidad de trabajo y outbox local.	Controladores con acceso directo al dominio; microservicios con transacciones distribuidas.	CU-01b, CU-04c, CU-06
03 · Exclusión de agenda y vencimientos	Retención temporal de 15 minutos y exclusión por tutor al crear, confirmar, liberar y extender; intervalos [inicio, fin).	Consultar y luego insertar sin exclusión.	BR-0001, 0006, 0012, 0015, 0018
04 · Resultado, seguimiento y disputa	Mantener separados el estado de ejecución, el seguimiento y la validez; una disputa oportuna reabre el seguimiento.	Bloquear toda disputa tras el cierre, o posponer siempre el cierre 24 horas.	BR-0002, 0007, 0010, 0013
05 · Cobro, desglose y liquidación	Pago registra 20/80 al confirmar; la liquidación usa el saldo vigente sin ventana ni revisión pendiente; PoliticaDeReembolso decide los importes.	Transferir el 80% de inmediato y tratar cada excepción como otro cobro.	BR-0004, 0005, 0008, 0009, 0012, 0017
06 · Resoluciones sustentadas e idempotentes	Disputa.resolver exige evidencia, justificación y responsable; resolución, auditoría y evento se guardan en la misma transacción.	Validar solo en la pantalla, o cerrar automáticamente a las 48 horas.	BR-0009, 0011, 0014, 0017, 0018, 0020
07 · Acuerdos, calificaciones y consultas	Calificacion y ReporteComentario fuera de Sesion; AcuerdoDeExtension y AcuerdoDeModalidad dentro de Sesion; promedio y dashboard desde proyecciones.	Valores sueltos en servicios o un booleano «ambos aceptan» enviado por un solo actor.	BR-0002, 0012, 0013, 0016, 0019

11. Trazabilidad

La matriz de trazabilidad, entregada como documento aparte, vincula las 20 necesidades, historias y reglas con la decisión, el componente y el criterio de aceptación que las respalda, y se puede recorrer en ambos sentidos. Ejemplos de cómo se lee:

Necesidad / regla	Decisión y componente	Dónde se comprueba
N1 · BR-0001 (15 minutos)	Reserva.confirmarPago y liberarPorTimeout; ADR-03	DS-02, mensajes de las fases 1 y 2
N6 · BR-0006 (sin traslapes)	ValidadorTraslapeHorario, usado al crear y al confirmar; ADR-03	DS-02, mensajes de verificarTraslape
N5 · BR-0005 (20/80)	Pago y DesglosePago; ADR-05	DS-02, confirmación del pago
N20 · BR-0020 (no cerrar sin evidencia)	Disputa.resolver y error EVIDENCIA_REQUERIDA; ADR-06	DS-01, bloque alt
N11 · BR-0011 (48 horas)	Disputa; resolución con responsable y auditoría; ADR-06	DS-01 y Etapa 4 (CU-04c)

Conclusiones

- **Domain Model fue la decisión que sostiene el resto.** Las reglas críticas (15 minutos, 50%, 20/80, traslapes y evidencia) tienen un solo responsable en el dominio, y eso evita que cada pantalla o proceso las reimplemente.
- **Cada capa tiene una frontera comprobable.** Los Application Services coordinan, los repositorios se definen en el dominio y se implementan en infraestructura, y el Unit of Work da una frontera transaccional clara con commit y rollback. Los dos diagramas de secuencia lo muestran con los mismos nombres del diagrama de clases.
- **CQRS se aplicó donde aporta.** Solo se separaron los dos Commands críticos y las dos Queries de lectura, sobre una sola base. Esto evita sobreingeniería y protege los datos internos del tutor.
- **SOLID encontró cambios reales**: dos reglas de los servicios pasaron al dominio y se propuso abrir PoliticaDeReembolso por causa.
- **El uso de IA aceleró borradores, pero cada resultado se verificó contra la Unidad 2**; varias sugerencias se corrigieron o descartaron, como colocar el cálculo del reembolso en el servicio o separar físicamente las bases de lectura y escritura. La bitácora de IA entregada aparte documenta esas iteraciones.

Decisiones pendientes de validación con el negocio

Antes de implementar deben aprobarse: los extremos exactos de los plazos de pago y de disputa; la zona horaria y el cómputo de los 30 días; el destino y cobro de la penalización del 50%; el reembolso tras una cancelación; qué ocurre si Soporte supera las 48 horas; el redondeo de importes; la escala y moderación de calificaciones; y las métricas, latencia y disponibilidad objetivo del dashboard. La arquitectura registra estos vacíos y no los resuelve por su cuenta.